Vehicle Speed Monitor System

Arduino Nano - Ultrasonic Obstacle Alert - RPM/Speed - Battery - Bluetooth Telemetry

A prototype vehicle dashboard built on an Arduino Nano that measures obstacle distance, wheel RPM and speed, battery percentage and total distance travelled, displays them on a 6-screen LCD menu (cycled with a touch sensor), auto-adjusts motor speed based on obstacle distance, and broadcasts telemetry over Bluetooth (HM-10) every 30 seconds.

1. How the System Works (Logic)

1.1 Obstacle Detection and Speed Control

The HC-SR04 ultrasonic sensor continuously measures distance ahead. Based on this distance (only while driving forward), the system picks one of three zones and scales the motor PWM accordingly:

Zone	Distance	Status LED	Motor Behaviour
Stopped	0 - 11 cm	Red ON	Motor forced to brake (PWM 255, both driver inputs HIGH), fast beeping alert
Warning	11 - 40 cm	Yellow ON	Speed capped - throttle mapped to a lower PWM range (0-120)
Safe	> 40 cm	Green ON	Full throttle range available (0-255)

1.2 RPM and Speed Calculation

A wheel-mounted IR/Hall sensor triggers an interrupt on every pulse (20 pulses per wheel revolution). Every 500 ms, the pulse count is converted to RPM, averaged over the last 3 readings for stability, and converted to speed in km/h using the wheel circumference. The same pulses accumulate distance travelled, which is saved to EEPROM every 5 seconds so total distance survives a power cycle.

1.3 Throttle and Direction Control

A potentiometer on A6 acts as the throttle, read as a 0-100% value. A dedicated touch sensor (Pin 13) toggles direction between FORWARD and REVERSE; in reverse, the obstacle-stop logic is bypassed (reversing away from an obstacle should not be blocked) and a periodic reverse beep sounds every 2 seconds as a safety warning to bystanders. If throttle exceeds 90%, a high-throttle double-beep alert fires once until throttle drops back below 85%.

1.4 Battery Monitoring

A voltage divider on A7 lets the Nano read the battery voltage (scaled x2 in code for a 2S pack). This is mapped from 6.0V-8.4V to a 0-100% battery percentage and shown on the dedicated battery screen along with the raw voltage.

1.5 LCD Menu (6 Screens)

A touch sensor on A1 cycles through 6 LCD screens on each tap, with a confirmation beep: (1) Distance + drive status, (2) RPM + throttle %, (3) animated direction mode (forward/reverse arrows), (4) battery % + voltage, (5) total distance in km, (6) speedometer in km/h.

1.6 Bluetooth Telemetry

Every 30 seconds, a full status report (time, drive status, distance, total km, speed, RPM, throttle %, battery % and voltage, direction) is sent over hardware Serial to an HM-10 Bluetooth module, viewable live on any BLE

serial terminal app.

2. System Block Diagram

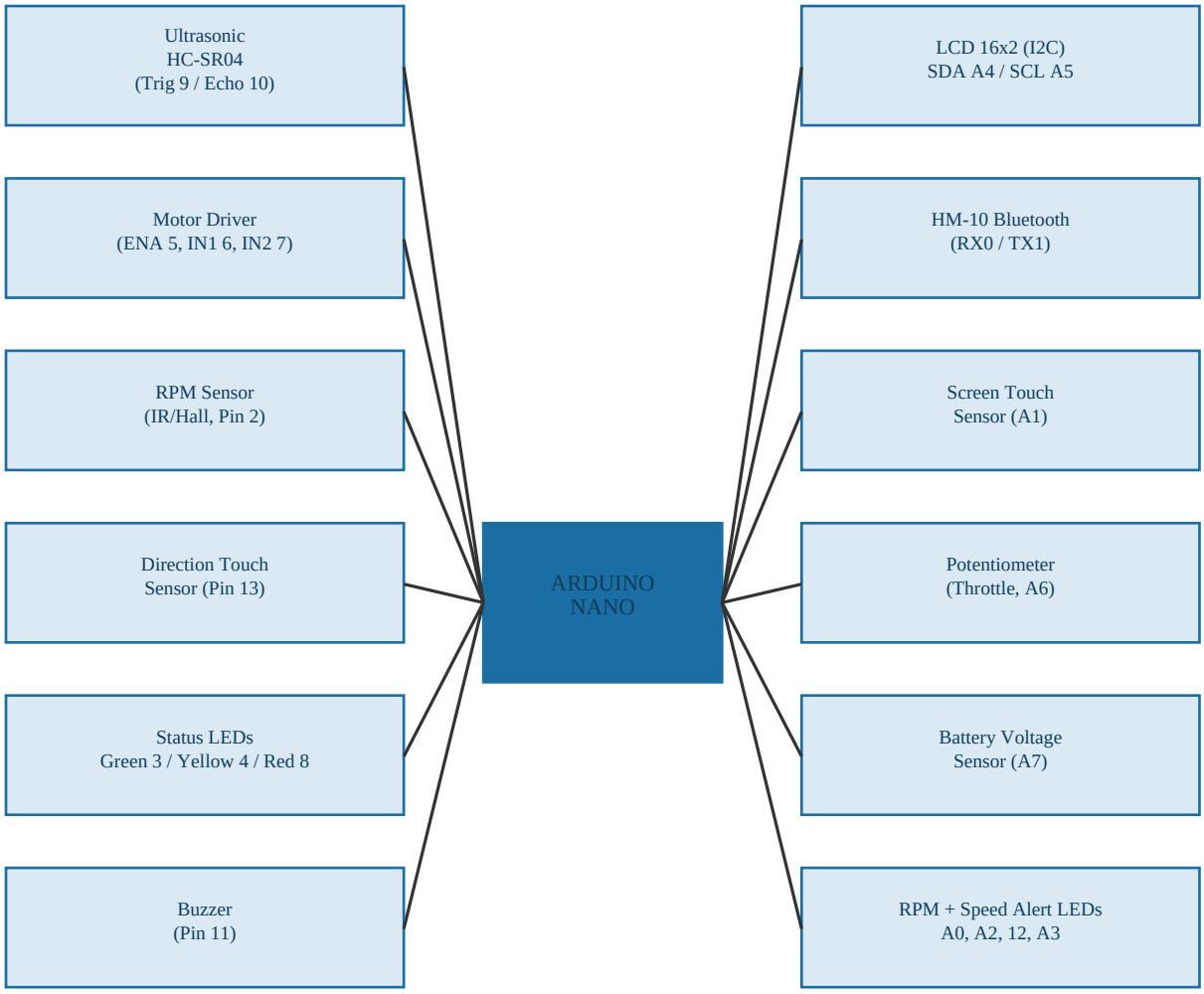

Figure 1: All peripherals connected to the central Arduino Nano

3. Connection Table

Component	Nano Pin	Notes
Ultrasonic HC-SR04 - Trig	D9	Digital output - trigger pulse
Ultrasonic HC-SR04 - Echo	D10	Digital input - echo pulse width
Motor Driver - ENA (PWM)	D5	analogWrite - motor speed
Motor Driver - IN1	D6	Direction control
Motor Driver - IN2	D7	Direction control
RPM Sensor (IR/Hall)	D2	INPUT_PULLUP, interrupt on FALLING
Green Status LED	D3	Digital output

Component	Nano Pin	Notes
Yellow Status LED	D4	Digital output
Red Status LED	D8	Digital output
Buzzer	D11	tone() output - all audio alerts
Red Speed Alert LED	D12	ON when throttle > 75%
Direction Touch Sensor	D13	Toggles FORWARD / REVERSE
RPM Alert Red LED	A0	ON when RPM > 300
Screen Touch Sensor	A1	Cycles the 6 LCD screens
RPM Alert Green LED	A2	ON when RPM <= 300
Yellow Speed Alert LED	A3	ON when throttle is 5-40%
LCD SDA (I2C)	A4	Default Nano I2C data
LCD SCL (I2C)	A5	Default Nano I2C clock
Throttle Potentiometer	A6	Analog input, 0-100%
Battery Voltage Sensor	A7	Analog input via voltage divider
HM-10 Bluetooth Module	RX0 / TX1	Hardware Serial @ 9600 baud

Note: LCD I2C address used in code is 0x27 - if your module doesn't respond, run an I2C scanner sketch and update the address (commonly 0x3F on some modules).

4. Working Prototype - Hardware Photos

Photos of the assembled breadboard prototype, showing each LCD screen live on the hardware.

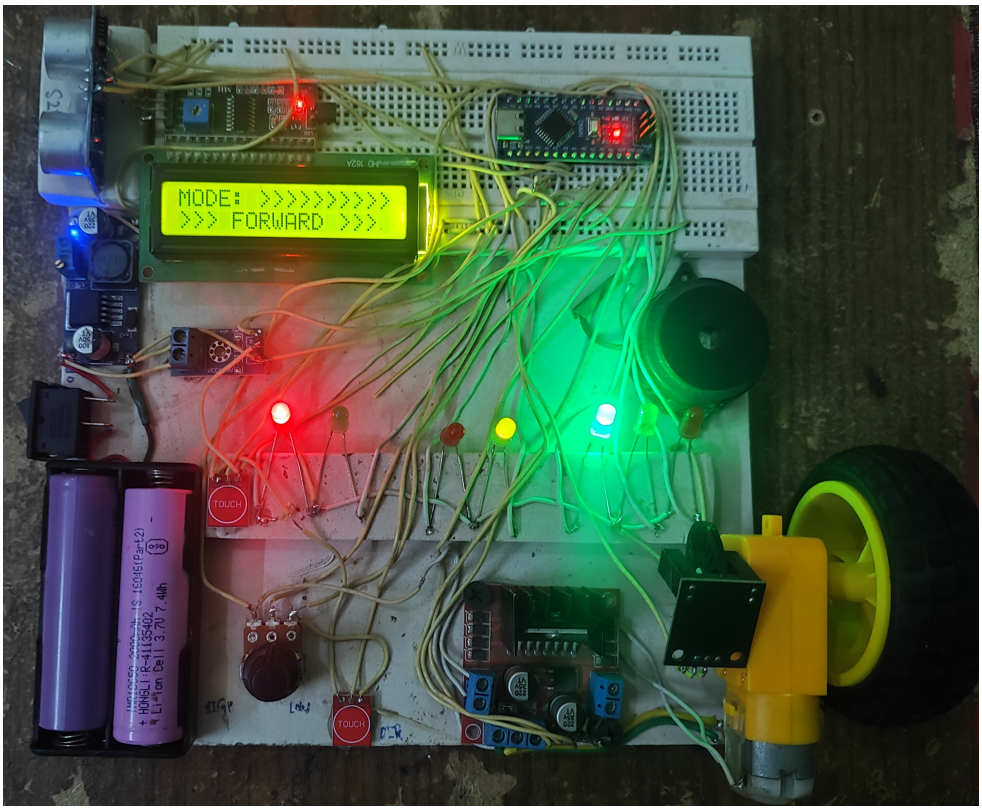

Figure 2: Screen 3 - Direction mode showing animated FORWARD arrows

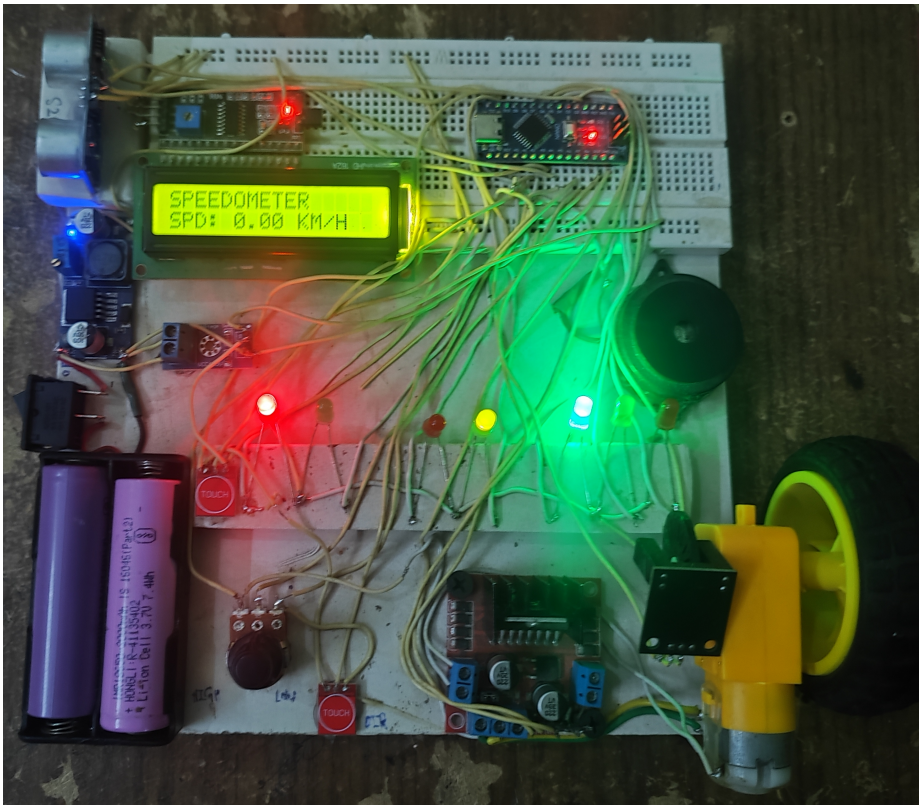

Figure 3: Screen 6 - Speedometer reading 0.00 km/h (stationary)

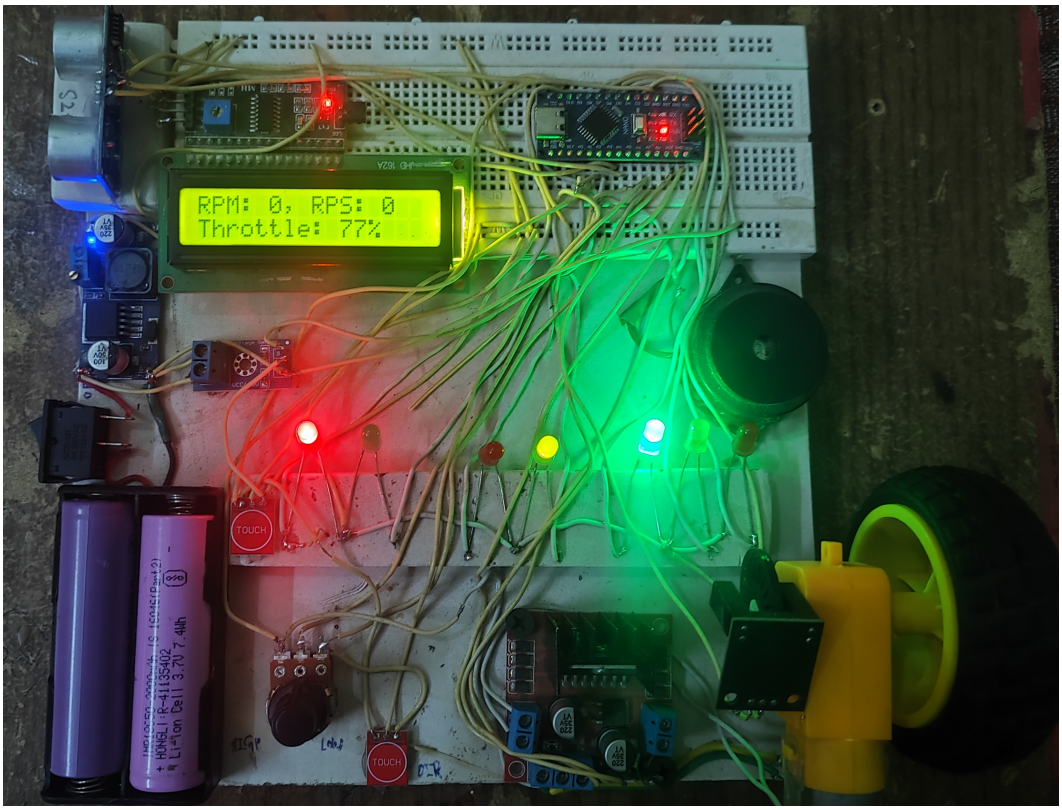

Figure 4: Screen 2 - RPM: 0, RPS: 0, Throttle at 77%

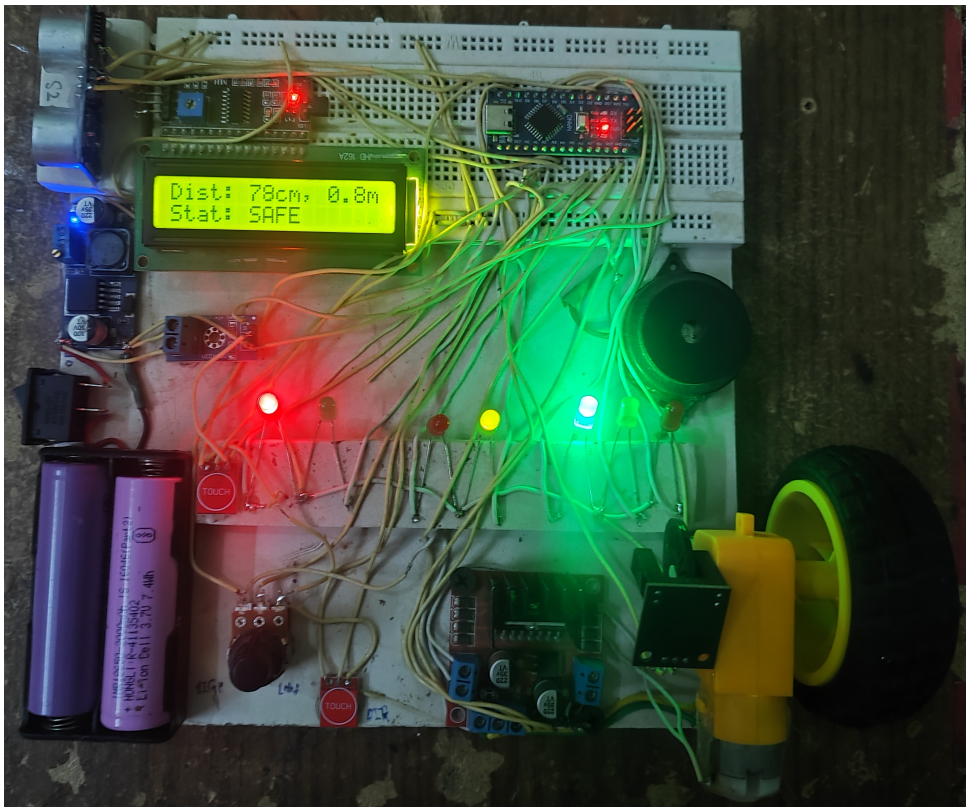

Figure 5: Screen 1 - Obstacle at 78cm, status SAFE (green zone)

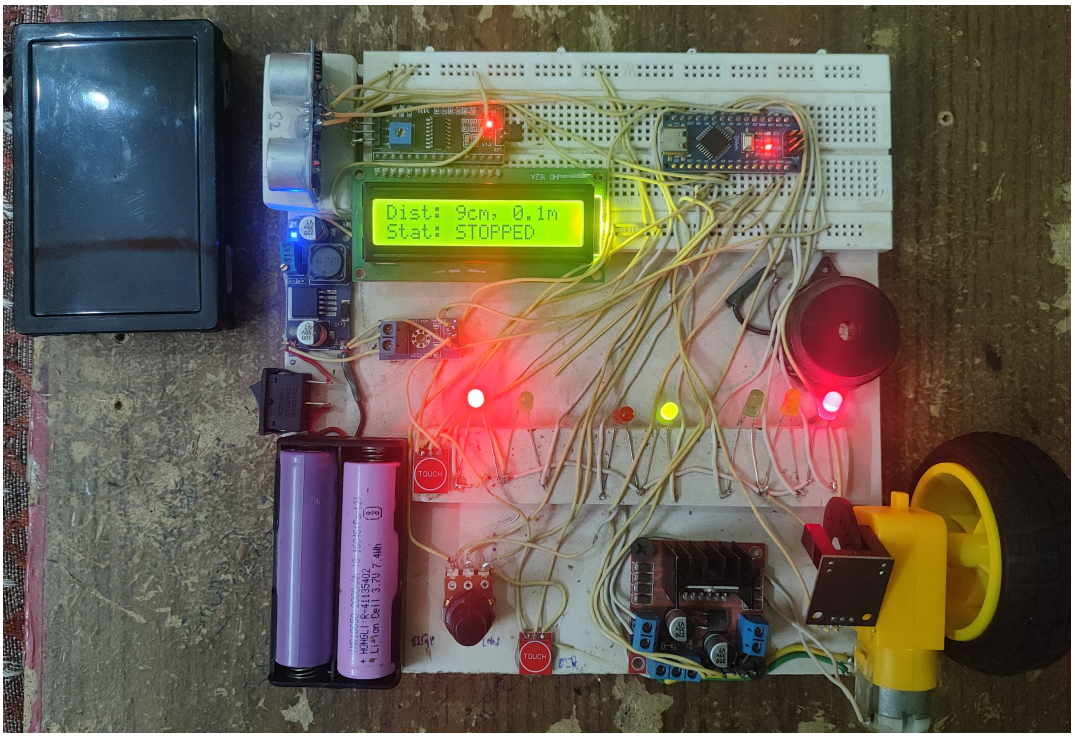

Figure 6: Screen 1 - Obstacle at 9cm, status STOPPED (red zone, motor braked)

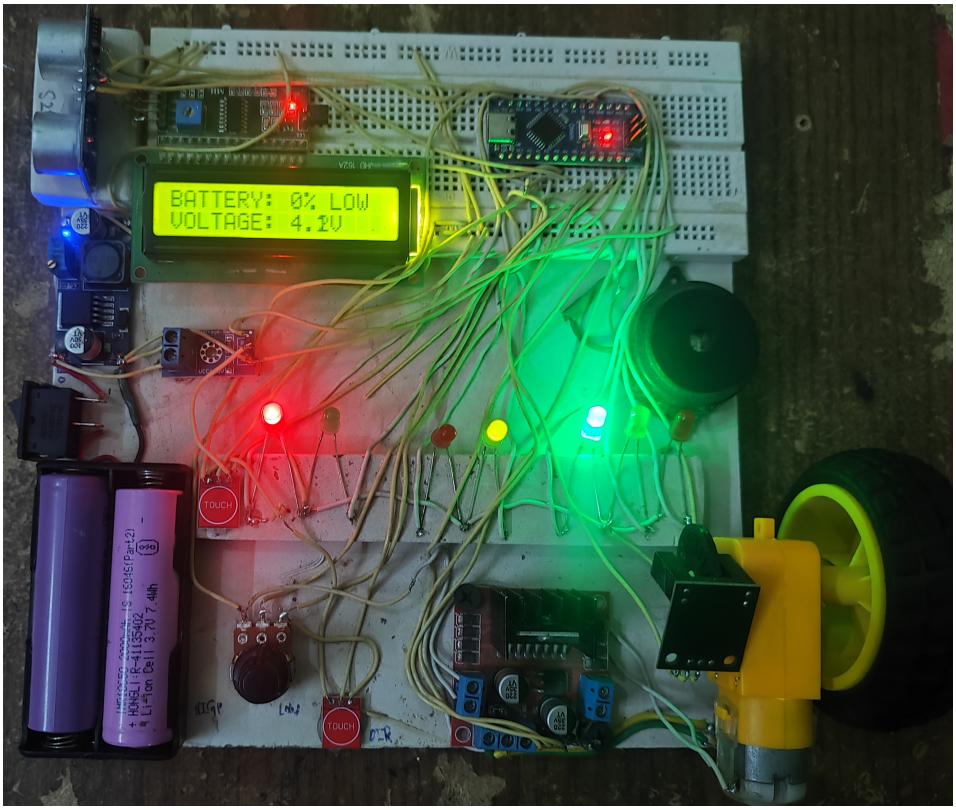

Figure 7: Screen 4 - Battery at 0% LOW, voltage 4.1V

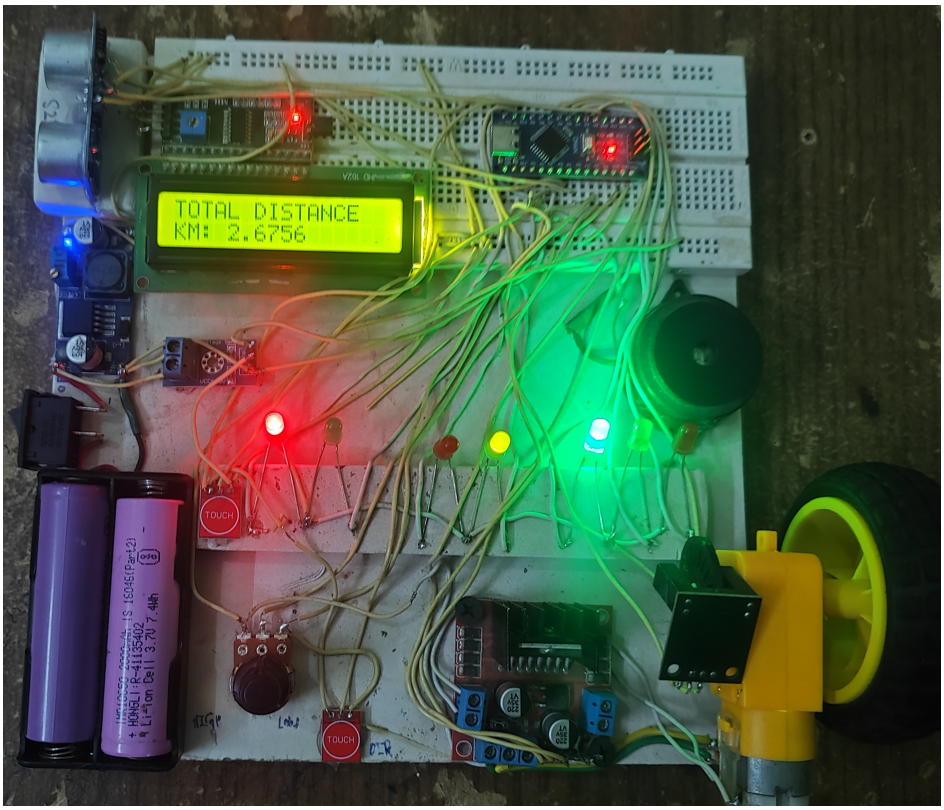

Figure 8: Screen 5 - Total distance travelled: 2.6756 km

5. Arduino Code (vehicle_speed_monitor.ino)

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <EEPROM.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

// Pin Definitions
#define trigPin 9
#define echoPin 10
#define ENA 5
#define IN1 6
#define IN2 7
#define rpmSensorPin 2
#define greenLED 3
#define yellowLED 4
#define redLED 8
#define screenBuzzerPin 11
#define rpmAlertGreenLED A2
#define rpmAlertRedLED A0
#define redSpeedAlertLED 12
#define yellowSpeedAlertLED A3
#define potPin A6
#define touchSensorPin A1
#define directionTouchPin 13
#define batteryPin A7

// --- Variables ---
float totalDistance = 0.0;
float currentSpeedKMH = 0.0;
const int EEPROM_ADDR = 0;
unsigned long lastEepromSave = 0;
const float WHEEL_CIRCUMFERENCE = 0.1885 / 1000.0;
#define PULSES_PER_REVOLUTION 20

volatile int pulseCount = 0;
volatile int totalPulsesForDist = 0;
String driveStatus = "Stopped";
int currentScreen = 1;
unsigned long lastRpmUpdate = 0;
int rpm = 0;
unsigned long lastPulseTime = 0;
unsigned long lastBuzzerMillis = 0;
int potValue = 0, potSpeed = 0, potPercentage = 0;
unsigned long previousLoopMillis = 0;
const unsigned long loopInterval = 100;
int lastTouchState = LOW, currentTouchState = LOW;
unsigned long lastDebounceTime = 0;
float lastValidDistance = 10;

float batteryVoltage = 0.0;
int batteryPercentage = 0;
int directionState = 0;
int lastDirectionTouchState = LOW;
unsigned long lastDirectionDebounceTime = 0;

bool highThrottleAlertTriggered = false;
unsigned long lastReverseBeepMillis = 0;
const unsigned long reverseBeepInterval = 2000;

int rpmValues[3] = {0};
int rpmIndex = 0;

unsigned long lastAnimUpdate = 0;
const unsigned long animInterval = 200;
int animFrame = 0;
const String forwardArrows = ">>>>>>>>>>>>>>";
const String reverseArrows = "<<<<<<<<<<<<<<";

// BLE Settings
unsigned long lastBLESend = 0;
const unsigned long bleInterval = 30000; // Har 30 second mein update

void setup() {
lcd.init();
lcd.backlight();

EEPROM.get(EEPROM_ADDR, totalDistance);
if (isnan(totalDistance) || totalDistance < 0) totalDistance = 0.0;

Serial.begin(9600); // HM-10 BLE

pinMode(trigPin, OUTPUT);
pinMode(echoPin, INPUT);
pinMode(ENA, OUTPUT);
pinMode(IN1, OUTPUT);
pinMode(IN2, OUTPUT);
pinMode(rpmSensorPin, INPUT_PULLUP);
pinMode(greenLED, OUTPUT);
```

```
pinMode(yellowLED, OUTPUT);
pinMode(redLED, OUTPUT);
pinMode(screenBuzzerPin, OUTPUT);
pinMode(rpmAlertGreenLED, OUTPUT);
pinMode(rpmAlertRedLED, OUTPUT);
pinMode(redSpeedAlertLED, OUTPUT);
pinMode(yellowSpeedAlertLED, OUTPUT);
pinMode(touchSensorPin, INPUT);
pinMode(directionTouchPin, INPUT);
pinMode(batteryPin, INPUT);

attachInterrupt(digitalPinToInterrupt(rpmSensorPin), countPulse, FALLING);

lcd.setCursor(0, 0);
lcd.print("VEHICLE MONITOR ");
lcd.setCursor(0, 1);
lcd.print("SYSTEM STARTING.");
delay(1500);

for(int i = 0; i < 3; i++) {
for(int freq = 600; freq <= 1200; freq += 100) {
tone(screenBuzzerPin, freq, 80);
delay(90);
}
delay(200);
}
noTone(screenBuzzerPin);

lcd.clear();
lcd.print("BLE Ready @10s");
delay(1000);
}

void loop() {
unsigned long currentMillis = millis();

if (currentMillis - previousLoopMillis >= loopInterval) {
previousLoopMillis = currentMillis;

noInterrupts();
int pulsesToProcess = totalPulsesForDist;
totalPulsesForDist = 0;
interrupts();

float distanceCovered = (pulsesToProcess / (float)PULSES_PER_REVOLUTION) * WHEEL_CIRCUMFERENCE;
totalDistance += distanceCovered;

if (currentMillis - lastEepromSave >= 5000) {
EEPROM.put(EEPROM_ADDR, totalDistance);
lastEepromSave = currentMillis;
}

int rawBatt = analogRead(batteryPin);
batteryVoltage = (rawBatt * 5.0 / 1023.0) * 2.0;
batteryPercentage = map(constrain(batteryVoltage * 10, 60, 84), 60, 84, 0, 100);

potValue = analogRead(potPin);
potPercentage = map(potValue, 0, 1023, 0, 100);

checkDirection(currentMillis);
float distance = getDistance();

// High Throttle Alert
if (potPercentage > 90 && !highThrottleAlertTriggered) {
tone(screenBuzzerPin, 1200, 80);
delay(100);
tone(screenBuzzerPin, 1200, 80);
highThrottleAlertTriggered = true;
} else if (potPercentage <= 85) {
highThrottleAlertTriggered = false;
}

// Reverse Beep
if (directionState == 1 && currentMillis - lastReverseBeepMillis >= reverseBeepInterval) {
lastReverseBeepMillis = currentMillis;
tone(screenBuzzerPin, 1500, 100);
delay(120);
tone(screenBuzzerPin, 1500, 100);
}

bool closeObstacle = (directionState == 0 && distance > 0 && distance <= 11);

if (directionState == 1) {
driveStatus = "REVERSE";
digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
potSpeed = map(potValue, 0, 1023, 0, 200);
analogWrite(ENA, potSpeed);
digitalWrite(redLED, HIGH); digitalWrite(yellowLED, LOW); digitalWrite(greenLED, LOW);
}
else {
```

```
if (closeObstacle) {
driveStatus = "STOPPED";
digitalWrite(IN1, HIGH); digitalWrite(IN2, HIGH);
analogWrite(ENA, 255);
digitalWrite(redLED, HIGH); digitalWrite(yellowLED, LOW); digitalWrite(greenLED, LOW);

if (currentMillis - lastBuzzerMillis >= 50) {
lastBuzzerMillis = currentMillis;
static bool beepState = false;
beepState = !beepState;
if (beepState) tone(screenBuzzerPin, 1200, 40);
else noTone(screenBuzzerPin);
}
}
else if (distance > 11 && distance <= 40) {
driveStatus = "WARNING";
digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
digitalWrite(redLED, LOW); digitalWrite(yellowLED, HIGH); digitalWrite(greenLED, LOW);
potSpeed = map(potValue, 0, 1023, 0, 120);
analogWrite(ENA, potSpeed);
noTone(screenBuzzerPin);
}
else {
driveStatus = "SAFE";
digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
digitalWrite(redLED, LOW); digitalWrite(yellowLED, LOW); digitalWrite(greenLED, HIGH);
potSpeed = map(potValue, 0, 1023, 0, 255);
analogWrite(ENA, potSpeed);
noTone(screenBuzzerPin);
}
}

if (!closeObstacle && rpm > 500) {
if (currentMillis - lastBuzzerMillis >= 80) {
lastBuzzerMillis = currentMillis;
static bool highPitch = true;
tone(screenBuzzerPin, highPitch ? 1200 : 1000, 70);
highPitch = !highPitch;
}
} else if (!closeObstacle) {
noTone(screenBuzzerPin);
}

updateRPM();
handleScreens(currentMillis);
sendDataToBLE(currentMillis); // BLE Data

digitalWrite(redSpeedAlertLED, (potPercentage > 75));
digitalWrite(yellowSpeedAlertLED, (potPercentage < 40 && potPercentage > 5));
}
}

// ==================== BLE DATA (Har 10 Second) ====================
void sendDataToBLE(unsigned long currentMillis) {
if (currentMillis - lastBLESend >= bleInterval) {
lastBLESend = currentMillis;

Serial.print("=== VEHICLE MONITOR ===\r\n");
Serial.print("Time : "); Serial.print(millis()/1000); Serial.print(" sec\r\n");
Serial.print("Status : "); Serial.print(driveStatus); Serial.print("\r\n");
Serial.print("Distance : "); Serial.print((int)lastValidDistance); Serial.print(" cm\r\n");
Serial.print("Total KM : "); Serial.print(totalDistance, 3); Serial.print(" km\r\n");
Serial.print("Speed : "); Serial.print(currentSpeedKMH, 1); Serial.print(" km/h\r\n");
Serial.print("RPM : "); Serial.print(rpm); Serial.print("\r\n");
Serial.print("Throttle : "); Serial.print(potPercentage); Serial.print("%\r\n");
Serial.print("Battery : "); Serial.print(batteryPercentage);
Serial.print("% ("); Serial.print(batteryVoltage, 1); Serial.print("V)\r\n");
Serial.print("Direction : "); Serial.print(directionState == 0 ? "FORWARD" : "REVERSE");
Serial.print("\r\n========================\r\n\r\n");
}
}

// ==================== Other Functions ====================
void handleScreens(unsigned long m) {
currentTouchState = digitalRead(touchSensorPin);

if (currentTouchState == HIGH && lastTouchState == LOW && (m - lastDebounceTime > 400)) {
lastDebounceTime = m;
tone(screenBuzzerPin, 1000, 100);
currentScreen = (currentScreen % 6) + 1;
lcd.clear();
animFrame = 0;
lastAnimUpdate = m;
}
lastTouchState = currentTouchState;

if (currentScreen == 1) {
lcd.setCursor(0, 0);
lcd.print("Dist: ");
lcd.print((int)lastValidDistance);
```

```
lcd.print("cm ");
lcd.print(lastValidDistance/100.0, 1);
lcd.print("m ");
lcd.setCursor(0, 1);
lcd.print("Stat: ");
lcd.print(driveStatus);
lcd.print(" ");
}
else if (currentScreen == 2) {
lcd.setCursor(0, 0);
lcd.print("RPM: "); lcd.print(rpm); lcd.print(" ");
lcd.setCursor(0, 1);
lcd.print("Throttle: "); lcd.print(potPercentage); lcd.print("% ");
}
else if (currentScreen == 3) {
lcd.setCursor(0, 0);
lcd.print("MODE: ");
if (m - lastAnimUpdate >= animInterval) {
lastAnimUpdate = m;
animFrame = (animFrame + 1) % 14;
}
String arrows = directionState == 0 ?
forwardArrows.substring(animFrame) + forwardArrows.substring(0, animFrame) :
reverseArrows.substring(animFrame) + reverseArrows.substring(0, animFrame);
lcd.setCursor(6, 0);
lcd.print(arrows.substring(0, 10));
lcd.setCursor(0, 1);
lcd.print(directionState == 0 ? ">>> FORWARD >>>" : "<<< REVERSE <<<");
}
else if (currentScreen == 4) {
String battStatus = batteryPercentage >= 95 ? "FULL" : batteryPercentage >= 70 ? "HIGH" : batteryPercentage >= 40 ? "MED " :
"LOW ";
lcd.setCursor(0, 0);
lcd.print("BAT: "); lcd.print(batteryPercentage); lcd.print("% "); lcd.print(battStatus);
lcd.setCursor(0, 1);
lcd.print("V: "); lcd.print(batteryVoltage, 1); lcd.print("V");
}
else if (currentScreen == 5) {
lcd.setCursor(0,0); lcd.print("TOTAL DISTANCE");
lcd.setCursor(0,1); lcd.print("KM: "); lcd.print(totalDistance, 4);
}
else if (currentScreen == 6) {
lcd.setCursor(0,0); lcd.print("SPEEDOMETER");
lcd.setCursor(0,1); lcd.print("SPD: "); lcd.print(currentSpeedKMH, 1); lcd.print(" km/h");
}
}

void checkDirection(unsigned long m) {
int dirTouch = digitalRead(directionTouchPin);
if (dirTouch == HIGH && lastDirectionTouchState == LOW && (m - lastDirectionDebounceTime > 400)) {
lastDirectionDebounceTime = m;
tone(screenBuzzerPin, 800, 100);
delay(150);
tone(screenBuzzerPin, 800, 100);
directionState = !directionState;
lcd.clear();
animFrame = 0;
}
lastDirectionTouchState = dirTouch;
}

void updateRPM() {
unsigned long currentMillis = millis();
if (currentMillis - lastRpmUpdate >= 500) {
lastRpmUpdate = currentMillis;
noInterrupts();
int currentRpm = (pulseCount * 120) / PULSES_PER_REVOLUTION;
pulseCount = 0;
interrupts();
rpmValues[rpmIndex % 3] = currentRpm;
rpmIndex++;
rpm = (rpmValues[0] + rpmValues[1] + rpmValues[2]) / 3;
currentSpeedKMH = (rpm * WHEEL_CIRCUMFERENCE * 60.0);
digitalWrite(rpmAlertRedLED, (rpm > 300));
digitalWrite(rpmAlertGreenLED, (rpm <= 300));
}
}

float getDistance() {
digitalWrite(trigPin, LOW); delayMicroseconds(2);
digitalWrite(trigPin, HIGH); delayMicroseconds(10);
digitalWrite(trigPin, LOW);
long duration = pulseIn(echoPin, HIGH, 25000);
if (duration == 0) return lastValidDistance;
float d = duration * 0.034 / 2;
if (d > 2 && d < 400) lastValidDistance = d;
return lastValidDistance;
}

void countPulse() {
```

```
unsigned long currentTime = millis();
if (currentTime - lastPulseTime > 5) {
pulseCount++;
totalPulsesForDist++;
lastPulseTime = currentTime;
}
}
```